

TRABAJO DE FIN DE GRADO

NeoCore: Sistema Integral de Reservas para Centro de Salud y Bienestar

Ciclo Formativo: Técnico Superior en Desarrollo de Aplicaciones Web (DAW)

Autor: Raúl López

Correo: raullopez20r@gmail.com

URL de producción: <https://neocoree.xyz>

Repositorio: https://github.com/Raullopez20/TFG_ADELANTADO

Año: 2026

ÍNDICE

1. Introducción y Motivación
 2. Análisis de Requisitos
 3. Diseño del Sistema
 4. Arquitectura Técnica
 5. Implementación del Backend
 6. Implementación del Frontend
 7. Sistema de Seguridad
 8. Sistema de Emails
 9. Gestión de Imágenes
 10. API REST — Documentación Completa
 11. Despliegue en Producción
 12. Pruebas y Validación
 13. Conclusiones y Trabajo Futuro
 14. Glosario
 15. Bibliografía y Referencias
-

1. Introducción y Motivación

1.1 Descripción del Proyecto

NeoCore es un sistema integral de gestión de reservas y citas para centros de salud y bienestar. El proyecto abarca el ciclo completo de una reserva: desde que un cliente elige un servicio y un profesional, hasta que la cita se confirma, se realiza y se deja una reseña.

El sistema está compuesto por: - Una **API REST** desarrollada con Django y Django REST Framework - Un **frontend** de usuario desarrollado con Next.js 16 (App Router) - Un **panel de administración** (backoffice) propio para gestionar todo el centro - Un **portal del profesional** para gestionar sus citas y horarios - Integración con servicios externos de **email** (Resend) e **imágenes** (Cloudinary) - Despliegue completo en **Vercel** + **Neon PostgreSQL** (arquitectura serverless)

1.2 Motivación

Los centros de salud y bienestar — fisioterapeutas, nutricionistas, psicólogos, entrenadores personales — siguen gestionando sus citas en muchos casos mediante llamadas telefónicas, agendas en papel o herramientas genéricas no adaptadas a sus necesidades. Esto genera:

- **Pérdida de tiempo** en gestión administrativa
- **Errores y solapamientos** de citas
- **Falta de visibilidad** para los clientes sobre la disponibilidad real
- **Ausencia de historial** digital de citas y pacientes

NeoCore nace para digitalizar y automatizar completamente este proceso, ofreciendo una solución moderna, segura y escalable adaptada específicamente al sector.

1.3 Objetivos

Objetivo principal:

Desarrollar una plataforma web completa y funcional que permita la gestión integral de reservas entre clientes y profesionales de la salud.

Objetivos específicos:

1. Implementar un sistema de autenticación seguro con roles diferenciados (cliente, profesional, administrador)
2. Desarrollar un motor de disponibilidad que calcule slots libres en tiempo real según el horario de cada profesional
3. Crear un backoffice administrativo con estadísticas, gestión de usuarios, servicios y profesionales
4. Implementar un sistema de notificaciones por email en cada cambio de estado de la reserva
5. Garantizar la seguridad de los datos aplicando las mejores prácticas (OWASP, GDPR, HTTPS)
6. Desplegar la aplicación en producción con una arquitectura serverless escalable
7. Desarrollar una API REST documentada y testeable mediante Swagger UI

1.4 Alcance

El proyecto incluye: - ☒ Registro y autenticación de usuarios - ☒ Catálogo público de servicios y profesionales - ☒ Sistema de reservas en 4 pasos con wizard interactivo - ☒ Gestión de disponibilidad semanal por profesional - ☒ Panel de administración completo - ☒ Portal del profesional para gestionar sus citas - ☒ Notificaciones email con plantillas HTML personalizadas - ☒ Almacenamiento de imágenes en la nube (Cloudinary) - ☒ Recuperación de contraseña por email - ☒ API REST documentada con Swagger/OpenAPI - ☒ Soporte multiidioma (español/inglés) - ☒ Diseño responsive (Mobile First) - ☒ Despliegue en producción en Vercel + Neon

No incluye (trabajo futuro): - ☒ Procesamiento de pagos online - ☒ App móvil nativa - ☒ Videoconsulta integrada

2. Análisis de Requisitos

2.1 Requisitos Funcionales

RF01 — Gestión de Usuarios

ID	Requisito
RF01.1	El sistema debe permitir el registro de nuevos usuarios con email, nombre, apellidos y contraseña
RF01.2	El registro debe requerir consentimiento GDPR explícito
RF01.3	El sistema debe autenticar usuarios mediante email y contraseña
RF01.4	El sistema debe emitir tokens JWT (access 15 min + refresh 7 días) al autenticarse
RF01.5	Los usuarios pueden actualizar su perfil: nombre, apellidos, teléfono, foto
RF01.6	Los profesionales pueden editar adicionalmente su especialidad y biografía
RF01.7	El usuario puede solicitar recuperación de contraseña por email
RF01.8	El administrador puede cambiar el rol de cualquier usuario
RF01.9	El administrador puede activar/desactivar cuentas de usuario

RF02 — Gestión de Servicios

ID	Requisito
RF02.1	El sistema debe mostrar públicamente el catálogo de servicios activos
RF02.2	Cada servicio tiene: nombre, descripción, duración en minutos, precio, imagen
RF02.3	Los servicios se asocian a profesionales que los pueden realizar
RF02.4	El administrador puede crear, editar y eliminar servicios
RF02.5	El administrador puede activar/desactivar servicios (sin eliminarlos)
RF02.6	Los servicios admiten imagen personalizada

RF03 — Sistema de Reservas

ID	Requisito
RF03.1	Solo usuarios con rol CLIENT pueden crear reservas
RF03.2	Una reserva vincula: cliente + profesional + servicio + fecha/hora
RF03.3	El sistema valida que el slot seleccionado esté dentro del horario del profesional
RF03.4	El sistema valida que no haya solapamiento con otras reservas
RF03.5	La reserva se crea en estado PENDING hasta que el profesional confirme

ID	Requisito
RF03.6	El profesional puede confirmar (→ CONFIRMED) o rechazar (→ REJECTED)
RF03.7	El profesional puede marcar una cita confirmada como completada (→ DONE)
RF03.8	El cliente puede cancelar reservas PENDING o CONFIRMED (→ CANCELED)
RF03.9	Se envía email al profesional cuando se crea una nueva reserva
RF03.10	Se envía email al cliente cuando la reserva es confirmada o rechazada

RF04 — Disponibilidad

ID	Requisito
RF04.1	Cada profesional define sus reglas de disponibilidad semanal (día + hora inicio + hora fin)
RF04.2	El sistema calcula los slots libres en un rango de fechas dado
RF04.3	Los slots ya reservados se excluyen automáticamente
RF04.4	El profesional puede definir períodos de ausencia (vacaciones, bajas)
RF04.5	Los slots durante ausencias se excluyen automáticamente

RF05 — Panel de Administración

ID	Requisito
RF05.1	El backoffice requiere rol ADMIN para acceder
RF05.2	El dashboard muestra KPIs: reservas hoy, pendientes, confirmadas, completadas
RF05.3	El dashboard muestra gráficos de reservas por servicio y profesional
RF05.4	El administrador puede gestionar todos los usuarios (roles, estado)
RF05.5	El administrador puede gestionar profesionales: crear, editar, foto, horario
RF05.6	El administrador puede gestionar servicios: crear, editar, imagen
RF05.7	La página de estadísticas muestra métricas para 7/30/90/365 días
RF05.8	El panel API Reference documenta todos los endpoints con prueba integrada

2.2 Requisitos No Funcionales

ID	Categoría	Requisito
RNF01	Seguridad	Las contraseñas se almacenan con hash Argon2 (nunca en texto plano)
RNF02	Seguridad	Los campos sensibles (teléfono, bio, notas) se encriptan con Fernet en DB
RNF03	Seguridad	Los tokens JWT tienen expiración corta (15 min access, 7 días refresh)
RNF04	Seguridad	Máximo 5 intentos fallidos de login antes de bloqueo (15 min)

ID	Categoría	Requisito
RNF05	Seguridad	Todos los endpoints implementan rate limiting
RNF06	Seguridad	La CSP restringe los orígenes de scripts, estilos e imágenes
RNF07	Rendimiento	Las páginas deben cargar en menos de 3 segundos
RNF08	Rendimiento	El backend usa paginación en listados (20 elementos/página)
RNF09	Disponibilidad	El sistema debe estar disponible 99.9% del tiempo
RNF10	Escalabilidad	La arquitectura serverless permite escalar automáticamente
RNF11	Usabilidad	El diseño es responsive y funciona en móvil, tablet y escritorio
RNF12	GDPR	Los usuarios pueden exportar y eliminar sus datos personales
RNF13	Internacionalización	La interfaz está disponible en español e inglés
RNF14	Accesibilidad	Los componentes UI cumplen WCAG 2.1 (Radix UI)

2.3 Casos de Uso Principales

CU01 — Reservar una Cita (Cliente)

Actor: Usuario CLIENT

Precondición: Usuario autenticado

Flujo principal:

1. El cliente accede a la página de reserva
2. Selecciona el servicio (tarjeta visual)
3. Selecciona el profesional (tarjeta con foto y especialidad)
4. Selecciona la fecha y la hora disponible
5. Añade notas opcionales
6. Confirma la reserva
7. El sistema crea la reserva en estado PENDING
8. Se envía email al profesional notificando la nueva solicitud
9. El cliente ve la pantalla de éxito con los detalles

Flujo alternativo (usuario no autenticado):

- 2a. Si el usuario no tiene cuenta, se guarda la selección en localStorage
- 2b. Se redirige al login/registro
- 2c. Tras autenticarse, se pre-rellena el formulario con la selección guardada

Postcondición: Booking creado con status=PENDING

CU02 — Confirmar/Rechazar Cita (Profesional)

Actor: Usuario PROFESSIONAL

Precondición: Existe una reserva PENDING asignada al profesional

Flujo principal (confirmar):

1. El profesional accede a "Mis Citas"
2. Ve la reserva con datos del cliente (nombre, email, teléfono)
3. Pulsa "Confirmar"
4. La reserva cambia a CONFIRMED
5. Se envía email al cliente confirmando la cita

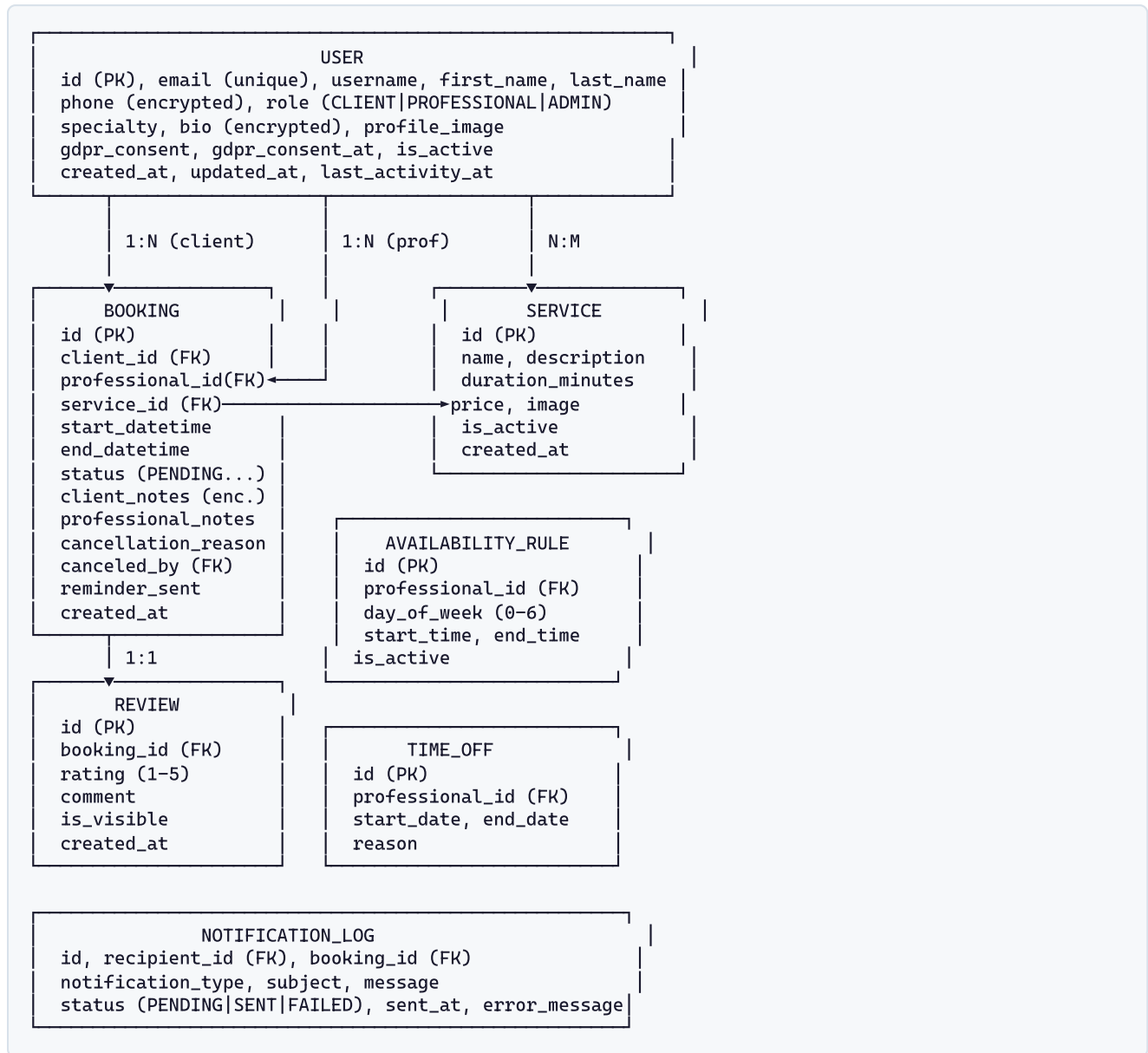
Flujo alternativo (rechazar):

- 3a. El profesional pulsa "Rechazar"
- 3b. Introduce motivo opcional del rechazo
- 3c. La reserva cambia a REJECTED
- 3d. Se envía email al cliente informando del rechazo

Postcondición: Booking con status=CONFIRMED o status=REJECTED

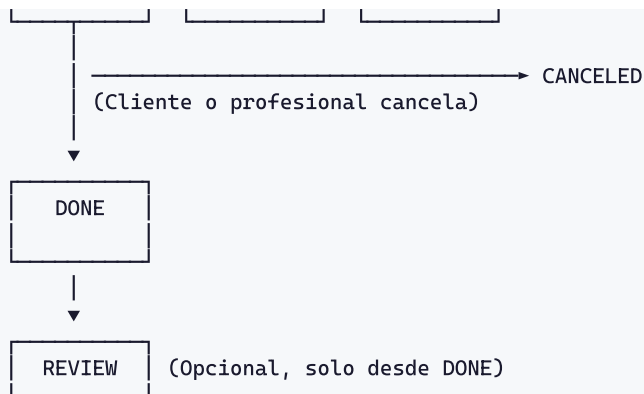
3. Diseño del Sistema

3.1 Diagrama Entidad-Relación

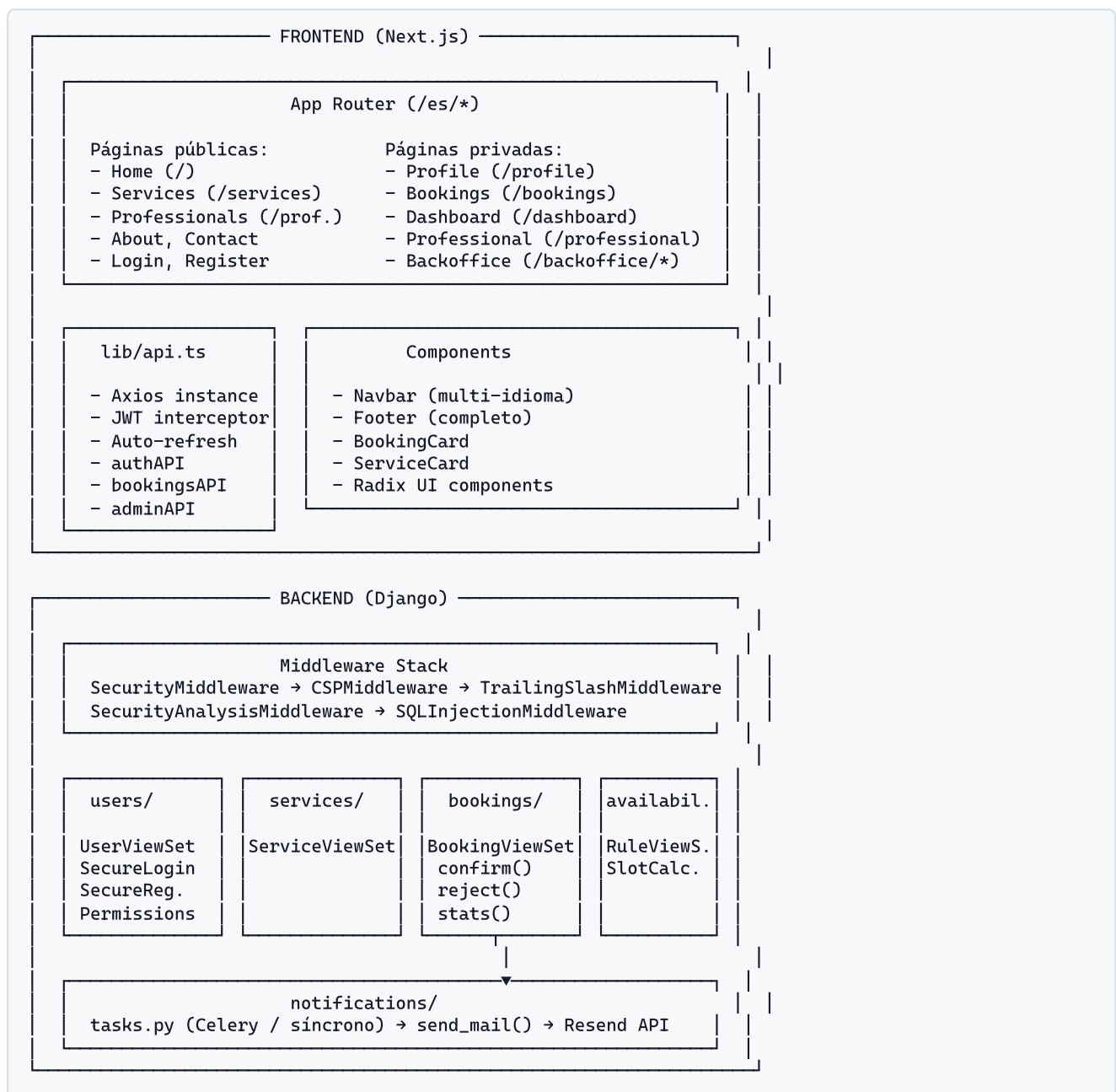


3.2 Máquina de Estados de una Reserva





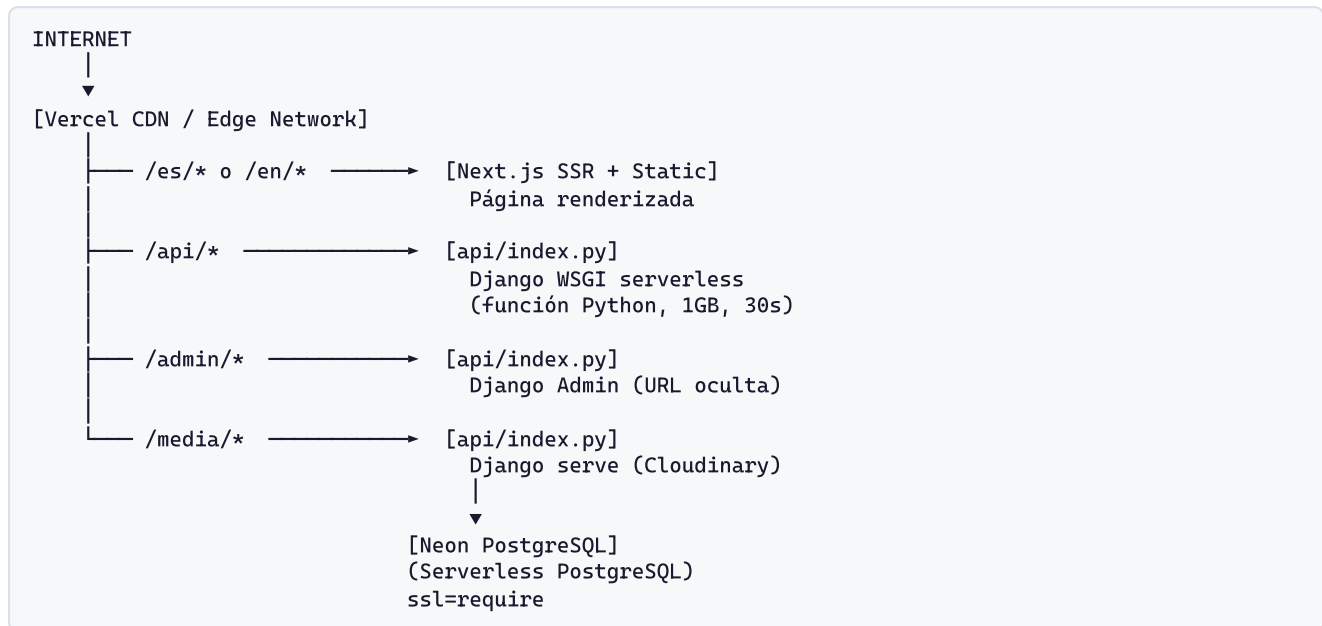
3.3 Diagrama de Componentes



4. Arquitectura Técnica

4.1 Arquitectura de Despliegue

NeoCore utiliza una arquitectura **serverless** desplegada completamente en **Vercel**, lo que elimina la necesidad de gestionar servidores.



4.2 El Punto de Entrada Serverless (api/index.py)

Uno de los desafíos más interesantes del proyecto fue adaptar Django (diseñado para servidores tradicionales) a un entorno serverless donde cada petición puede ser una función nueva ("cold start").

```
# api/index.py - Entrada WSGI para Vercel
"""
En cada cold start de Vercel (nueva instancia de la función):
1. Se importa y configura Django
2. Se ejecutan las migraciones pendientes (idempotente)
3. Se guarda el resultado en caché en memoria
4. La función WSGI queda lista para servir peticiones
"""

import os, sys, traceback
from pathlib import Path

# Configura el path para que Django encuentre sus módulos
_repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
_backend_root = os.path.join(_repo_root, "PROYECTO-NEOCORE", "neocore", "backend")
sys.path.insert(0, _backend_root)
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "neocore.settings")

_startup_error = None
_django_app = None

try:
    import django
    django.setup()

    # Auto-migración en cold start (idempotente y rápida si no hay cambios)
```

```

from django.core.management import call_command
from django.db.migrations.exceptions import InconsistentMigrationHistory
from io import StringIO

def _run_migrate():
    buf = StringIO()
    call_command("migrate", "--run-syncdb", interactive=False, verbosity=2, stdout=buf)
    return buf.getvalue()

try:
    _run_migrate()
except InconsistentMigrationHistory:
    # Fix para inconsistencias de historial (caso especial con allauth)
    from django.db import connection
    with connection.cursor() as cur:
        cur.execute("DELETE FROM django_migrations WHERE app IN ('account', 'socialaccount')")
    _run_migrate()

from django.core.wsgi import get_wsgi_application
_django_app = get_wsgi_application()

except Exception:
    _startup_error = traceback.format_exc()

def app(enviro, start_response):
    """Punto de entrada WSGI. Vercel llama a esta función por cada petición."""
    if _django_app is None:
        # Error en el arranque: muestra información diagnóstica
        body = b"Django startup failed:\n\n" + _startup_error.encode("utf-8", errors="replace")
        start_response("500 Internal Server Error", [
            ("Content-Type", "text/plain"),
            ("Content-Length", str(len(body))),
        ])
        return [body]
    return _django_app(enviro, start_response)

handler = app # Vercel también acepta 'handler' como nombre

```

Problemas resueltos:

1. **Vercel strip trailing slashes:** Vercel CDN elimina el `/` final de las URLs antes de pasarlas a Django, lo que causaba que las URLs de Django (que esperan `/`) no encontraran nada. Solución: `TrailingSlashMiddleware` que reagrega el slash internamente sin HTTP redirect.
2. **InconsistentMigrationHistory:** En la fase inicial del proyecto, allauth tenía sus migraciones registradas antes que el modelo User personalizado. Solución en el cold start: detectar el error y eliminar las entradas conflictivas del historial.
3. **Imágenes sin servidor:** En Vercel el filesystem es efímero (`/tmp/`). Solución: rewrite `/media/*` → Django + integración con Cloudinary para almacenamiento permanente.

4.3 El Middleware TrailingSlashMiddleware (Problema y Solución)

Este middleware resuelve un problema específico de la combinación Vercel + Django:

```

# PROBLEMA:
# 1. Browser solicita: POST /api/auth/login/
# 2. Vercel CDN hace 308 redirect a: /api/auth/login (sin slash)
# 3. Browser re-hace POST a: /api/auth/login
# 4. Django busca URL "api/auth/login" (sin slash)
# 5. Django no encuentra → 404

# SOLUCIÓN: Reescritura interna (sin redirect HTTP)
class TrailingSlashMiddleware(MiddlewareMixin):

```

```

"""
Reagrega el trailing slash INTERNAMENTE.

Diferencia con APPEND_SLASH=True: ese hace un 301/302 redirect HTTP.
Con Vercel que ya hizo un 308, tendríamos un bucle.
Este middleware modifica path_info directamente, sin respuesta HTTP.
"""
def process_request(self, request):
    path = request.path
    last_segment = path.split('/')[-1]
    # Solo afecta rutas sin extensión (no archivos .css, .js, etc.)
    if path != '/' and not path.endswith('/') and '.' not in last_segment:
        request.path_info = path + '/'
        request.META['PATH_INFO'] = path + '/'
    return None

```

4.4 Configuración de vercel.json

```

{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "framework": "nextjs",
  "installCommand": "npm install && cd PROYECTO_NEOCORE/neocore/frontend && npm install",
  "buildCommand": "cd PROYECTO_NEOCORE/neocore/frontend && npm run build",
  "outputDirectory": "PROYECTO_NEOCORE/neocore/frontend/.next",
  "functions": {
    "api/index.py": {
      "maxDuration": 30,
      "memory": 1024
    }
  },
  "rewrites": [
    { "source": "/api/:path*", "destination": "/api/index" },
    { "source": "/admin/:path*", "destination": "/api/index" },
    { "source": "/media/:path*", "destination": "/api/index" }
  ],
  "headers": [
    {
      "source": "/*",
      "headers": [
        { "key": "Strict-Transport-Security", "value": "max-age=31536000; includeSubDomains; preload" },
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "X-Frame-Options", "value": "DENY" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
        { "key": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=()" },
        { "key": "X-XSS-Protection", "value": "1; mode=block" }
      ]
    }
  ]
}

```

Puntos clave del vercel.json: - `framework: nextjs` → Vercel detecta y despliega el frontend Next.js - `functions` → La función Python tiene 30s de timeout y 1GB de memoria - `rewrites` → Redirige `/api/*`, `/admin/*` y `/media/*` a Django - `headers` → Cabeceras de seguridad HTTP aplicadas a todas las respuestas

5. Implementación del Backend

5.1 Modelo de Usuario Personalizado

Extender el modelo de usuario de Django fue una de las primeras decisiones críticas del proyecto. Django permite esto mediante `AbstractUser` o `AbstractBaseUser`. Se eligió `AbstractUser` porque mantiene toda la funcionalidad de Django (permisos, admin) y solo añade los campos necesarios.

```
# apps/users/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models
from encrypted_model_fields.fields import EncryptedCharField, EncryptedTextField

class User(AbstractUser):
    """
    Modelo de usuario personalizado que usa email como identificador
    principal en lugar de username. Soporta tres roles:
    CLIENT, PROFESSIONAL y ADMIN.
    """

    class Role(models.TextChoices):
        CLIENT = 'CLIENT', 'Cliente'
        PROFESSIONAL = 'PROFESSIONAL', 'Profesional'
        ADMIN = 'ADMIN', 'Administrador'

    # Email como identificador único (no username)
    email = models.EmailField(unique=True)
    username = models.CharField(max_length=150, unique=True, blank=True, null=True)

    # Información personal
    first_name = models.CharField(max_length=150)
    last_name = models.CharField(max_length=150)

    # Teléfono encriptado en la base de datos
    phone = EncryptedCharField(max_length=20, blank=True, default='')

    # Rol del sistema
    role = models.CharField(max_length=20, choices=Role.choices, default=Role.CLIENT)

    # Campos de profesional
    specialty = models.CharField(max_length=100, blank=True)
    bio = EncryptedTextField(blank=True, default='') # Biografía encriptada
    profile_image = models.ImageField(upload_to='profiles/', blank=True, null=True)

    # Auditoría
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    last_activity_at = models.DateTimeField(auto_now=True)

    # GDPR
    gdpr_consent = models.BooleanField(default=False)
    gdpr_consent_at = models.DateTimeField(null=True, blank=True)
    is_active = models.BooleanField(default=True)

    # Django usa email para autenticar
    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['first_name', 'last_name']

    # Propiedades de rol (más legible que comparar strings)
    @property
    def is_client(self):
        return self.role == self.Role.CLIENT

    @property
    def is_professional(self):
```

```

        return self.role == self.Role.PROFESSIONAL

    @property
    def is_admin_role(self):
        return self.role == self.Role.ADMIN

    def save(self, *args, **kwargs):
        # Auto-generar username desde email (requerido por Django)
        if not self.username:
            self.username = self.email.split('@')[0][:150]
        super().save(*args, **kwargs)

    def get_full_name(self):
        return f"{self.first_name} {self.last_name}".strip()

    def __str__(self):
        return f"{self.email} ({self.role})"

```

Por qué `EncryptedCharField` y `EncryptedTextField`?

La biblioteca `django-encrypted-model-fields` usa **cifrado simétrico Fernet** (AES-128-CBC con HMAC-SHA256) para cifrar los valores antes de almacenarlos en la base de datos. La clave de cifrado se configura en `FIELD_ENCRYPTION_KEY` o se deriva del `SECRET_KEY` de Django.

Esto significa que aunque un atacante tuviera acceso directo a la base de datos PostgreSQL, los valores de teléfono, biografía y notas de cita aparecerían como texto ilegible.

5.2 Sistema de Permisos

```

# apps/users/permissions.py
from rest_framework.permissions import BasePermission

class IsAdmin(BasePermission):
    """Solo permite acceso a usuarios con rol ADMIN."""
    def has_permission(self, request, view):
        return (
            request.user and
            request.user.is_authenticated and
            (request.user.is_admin_role or request.user.is_staff)
        )

class IsOwnerOrAdmin(BasePermission):
    """
    Permite acceso al propietario del objeto o a administradores.
    Usado en vistas de perfil de usuario.
    """
    def has_object_permission(self, request, view, obj):
        if request.user.is_admin_role or request.user.is_staff:
            return True
        # El objeto tiene un campo 'user' o es el propio user
        return obj == request.user or getattr(obj, 'user', None) == request.user

```

5.3 ViewSet de Usuarios con Permisos Dinámicos

```

# apps/users/views.py
class UserViewSet(viewsets.ModelViewSet):
    """
    ViewSet de usuarios. Los permisos varían según la acción:
    - list, create, destroy: solo admin
    - update, retrieve: propietario o admin
    - professionals, professional_detail: público (AllowAny)
    """
    queryset = User.objects.all()
    permission_classes = [IsAuthenticated]

```

```

filter_backends = [DjangoFilterBackend, filters.SearchFilter, filters.OrderingFilter]
filterset_fields = ['role', 'specialty', 'is_active']
search_fields = ['first_name', 'last_name', 'email', 'specialty']
ordering_fields = ['created_at', 'first_name', 'last_name']
ordering = ['-created_at']

def get_serializer_class(self):
    """
    Selecciona el serializer según la acción y el rol del usuario.
    Admin → AdminUserSerializer (todos los campos, incluido role y password)
    Usuario normal en update → UserUpdateSerializer (campos limitados)
    Resto → UserSerializer o AdminUserSerializer
    """
    if getattr(self, 'swagger_fake_view', False):
        return UserSerializer
    is_admin = hasattr(self.request.user, 'is_admin_role') and (
        self.request.user.is_admin_role or self.request.user.is_staff
    )
    if self.action in ['update', 'partial_update']:
        return AdminUserSerializer if is_admin else UserUpdateSerializer
    if is_admin:
        return AdminUserSerializer
    return UserSerializer

def get_permissions(self):
    """Permisos dinámicos según la acción."""
    if self.action == 'professionals':
        return [AllowAny()]
    if self.action in ['list', 'create', 'destroy']:
        return [IsAdmin()]
    if self.action in ['update', 'partial_update', 'retrieve']:
        return [IsOwnerOrAdmin()]
    return super().get_permissions()

def get_queryset(self):
    """Admin ve todos; resto solo se ve a sí mismo."""
    if getattr(self, 'swagger_fake_view', False):
        return User.objects.none()
    user = self.request.user
    if hasattr(user, 'is_admin_role') and (user.is_admin_role or user.is_staff):
        return User.objects.all()
    return User.objects.filter(id=user.id)

@action(detail=False, methods=['get'], permission_classes=[IsAuthenticated])
def me(self, request):
    """GET /api/auth/users/me/ - Perfil del usuario autenticado."""
    serializer = UserSerializer(request.user, context={'request': request})
    return Response(serializer.data)

@action(detail=False, methods=['patch'], permission_classes=[IsAuthenticated])
def update_me(self, request):
    """PATCH /api/auth/users/update_me/ - Actualizar mi perfil."""
    serializer = UserUpdateSerializer(
        request.user,
        data=request.data,
        partial=True,
        context={'request': request}
    )
    serializer.is_valid(raise_exception=True)
    serializer.save()
    return Response(UserSerializer(request.user, context={'request': request}).data)

@action(detail=False, methods=['get'], permission_classes=[AllowAny])
def professionals(self, request):
    """GET /api/auth/users/professionals/ - Listado público de profesionales."""
    queryset = User.objects.filter(role=User.Role.PROFESSIONAL, is_active=True)
    specialty = request.query_params.get('specialty')
    if specialty:
        queryset = queryset.filter(specialty__icontains=specialty)
    serializer = ProfessionalSerializer(queryset, many=True, context={'request': request})
    return Response(serializer.data)

```


5.4 Motor de Disponibilidad

El cálculo de slots disponibles es la funcionalidad más compleja del backend. El algoritmo:

1. Obtiene las reglas de disponibilidad del profesional para el rango de fechas solicitado
2. Genera todos los slots posibles según la duración del servicio
3. Excluye slots ocupados por reservas existentes (PENDING o CONFIRMED)
4. Excluye slots durante períodos de ausencia (TimeOff)
5. Excluye slots en el pasado

```
# apps/availability/services.py
from datetime import datetime, timedelta, date, time
from django.utils import timezone
from .models import AvailabilityRule, TimeOff
from apps.bookings.models import Booking

class AvailabilityService:
    """
    Servicio que calcula los slots disponibles para un profesional.

    Ejemplo de uso:
        slots = AvailabilityService.get_available_slots(
            professional_id=1,
            service_duration=60,      # minutos
            start_date='2026-06-10',
            end_date='2026-06-15'
        )
    """

    @staticmethod
    def get_available_slots(professional_id: int, service_duration: int,
                           start_date: date, end_date: date) -> list:
        """
        Retorna lista de slots disponibles (datetime inicio + fin).
        Un slot es disponible si:
        1. Está dentro del horario del profesional (AvailabilityRule)
        2. No está ocupado por otra reserva (PENDING o CONFIRMED)
        3. No cae en un período de ausencia (TimeOff)
        4. No es en el pasado
        """
        now = timezone.now()
        slots = []

        # Obtener reglas de disponibilidad activas del profesional
        rules = AvailabilityRule.objects.filter(
            professional_id=professional_id,
            is_active=True,
        )
        rules_by_day = {r.day_of_week: r for r in rules}

        # Obtener ausencias que se solapan con el rango de fechas
        time_offs = TimeOff.objects.filter(
            professional_id=professional_id,
            start_date__lte=end_date,
            end_date__gte=start_date,
        )
        time_off_dates = set()
        for to in time_offs:
            d = to.start_date
            while d <= to.end_date:
                time_off_dates.add(d)
                d += timedelta(days=1)

        # Obtener reservas existentes en el rango
        existing_bookings = Booking.objects.filter(
            professional_id=professional_id,
            status__in=[Booking.Status.PENDING, Booking.Status.CONFIRMED],
```

```

        start_datetime__date__range=(start_date, end_date),
    ).values_list('start_datetime', 'end_datetime')

    # Iterar día a día
    current_date = start_date
    while current_date <= end_date:
        # ¿Tiene regla para este día de la semana?
        day_of_week = current_date.weekday() # 0=Lunes, 6=Domingo
        if day_of_week not in rules_by_day:
            current_date += timedelta(days=1)
            continue

        # ¿Está en ausencia?
        if current_date in time_off_dates:
            current_date += timedelta(days=1)
            continue

        rule = rules_by_day[day_of_week]

        # Generar slots cada 'service_duration' minutos
        slot_start = datetime.combine(current_date, rule.start_time)
        slot_start = timezone.make_aware(slot_start)
        day_end = datetime.combine(current_date, rule.end_time)
        day_end = timezone.make_aware(day_end)

        while slot_start + timedelta(minutes=service_duration) <= day_end:
            slot_end = slot_start + timedelta(minutes=service_duration)

            # ¿Es en el pasado?
            if slot_start <= now:
                slot_start = slot_end
                continue

            # ¿Se solapa con alguna reserva existente?
            overlaps = any(
                b_start < slot_end and b_end > slot_start
                for b_start, b_end in existing_bookings
            )

            if not overlaps:
                slots.append({
                    'start_datetime': slot_start.isoformat(),
                    'end_datetime': slot_end.isoformat(),
                    'is_available': True,
                })

            slot_start = slot_end

        current_date += timedelta(days=1)

    return slots

```

5.5 Serializer con Validaciones de Seguridad

```

# apps/users/serializers.py
import re
from bleach import clean
from rest_framework import serializers

NAME_REGEX = re.compile(r"^[a-zA-ZáéíóúÁÉÍÓÚñÑüÜ\s\-'"]+$")

class CustomRegisterSerializer(RegisterSerializer):
    """
    Serializer de registro con validaciones estrictas de seguridad.
    Extiende el RegisterSerializer de dj-rest-auth.
    """
    first_name = serializers.CharField(required=True, max_length=150)
    last_name = serializers.CharField(required=True, max_length=150)
    phone = serializers.CharField(required=False, max_length=20)

```

```

gdpr_consent = serializers.BooleanField(required=True)

def validate_email(self, value):
    """
    Normaliza y valida el email. Verifica unicidad directamente
    en el modelo User (no en EmailAddress de allauth, que puede
    estar vacía con ACCOUNT_EMAIL_VERIFICATION='none').
    """
    value = value.lower().strip()
    EmailValidator()(value)
    if get_user_model().objects.filter(email__iexact=value).exists():
        raise serializers.ValidationError('Ya existe un usuario registrado con este email.')
    return value

def validate_first_name(self, value):
    """
    Valida que el nombre solo contenga letras (incluye acentos y ñ).
    Sanitiza con Bleach para eliminar cualquier HTML/script.
    """
    value = value.strip()
    if not NAME_REGEX.match(value):
        raise serializers.ValidationError('El nombre contiene caracteres no permitidos.')
    return clean(value, tags=[], attributes={}, strip=True)

def validate_gdpr_consent(self, value):
    if value is not True:
        raise serializers.ValidationError('Debes aceptar el consentimiento RGPD para
registrarte.')
    return value

```

5.6 Vistas de Autenticación Seguras

Uno de los bugs más interesantes encontrados durante el desarrollo fue que `@method_decorator(ratelimit(...), name='dispatch')` de `django-ratelimit` rompía el pipeline de respuesta de DRF, causando `AssertionError: .accepted_renderer not set on Response`.

Causa: El decorador `method_decorator(ratelimit, name='dispatch')` modifica el método `dispatch` de la clase antes de que DRF pueda ejecutar `finalize_response()`, que es quien establece el `renderer`. Sin `renderer`, DRF no puede serializar la respuesta.

Solución: Usar el sistema de throttling nativo de DRF en lugar de `django-ratelimit`:

```

# apps/users/security_views.py
from rest_framework.throttling import AnonRateThrottle

class LoginRateThrottle(AnonRateThrottle):
    """
    Limita los intentos de login a 5 por hora por IP.
    Usa el sistema de throttling nativo de DRF, que se integra
    correctamente con finalize_response() y el pipeline de renderers.
    """
    scope = 'login' # → DEFAULT_THROTTLE_RATES['login'] = '5/hour'

class SecureLoginView(LoginView):
    """Login con rate limiting DRF-nativo y logging de seguridad."""
    throttle_classes = [LoginRateThrottle]

    def throttled(self, request, wait):
        """Se ejecuta cuando se supera el límite. Registra el evento."""
        security_logger.warning('ratelimit.login.blocked', extra={
            'event': {
                'timestamp': datetime.now(timezone.utc).isoformat(),
                'ip': _get_client_ip(request),
                'path': request.path,
                'wait_seconds': wait,
            }
        })

```

```

    })
    raise Throttled(wait, detail='Demasiados intentos de inicio de sesión. Espera e inténtalo de nuevo.')
```

class SecurePasswordResetView(PasswordResetView):

"""
 Recuperación de contraseña sin CSRF y con manejo de fallos de email.

 Problema original: SessionAuthentication en DEFAULT_AUTHENTICATION_CLASSES causaba que, si el navegador tenía una cookie de sesión, se exigía CSRF. Como el frontend usa JWT (no cookies), nunca enviaba X-CSRFToken → 403.

 Solución: authentication_classes = [] elimina SessionAuthentication para esta vista pública específica.
 """

authentication_classes = [] # Sin autenticación → sin CSRF enforcement
 permission_classes = [AllowAny]
 serializer_class = FrontendPasswordResetSerializer

def post(self, request, *args, **kwargs):

try:

return super().post(request, *args, **kwargs)

except Exception:

Si el SMTP falla, devolvemos 200 igualmente.
 # No revelamos si el email existe en el sistema (seguridad).
 pass

return Response(
 {'detail': 'Si el correo existe, recibirás las instrucciones en breve.'},
 status=status.HTTP_200_OK,
)

5.7 Señales para Notificaciones de Reservas

apps/bookings/signals.py

"""
 Las señales post_save detectan cambios de estado en las reservas y envían notificaciones por email.

 En Vercel no hay Celery/Redis, por lo que el envío es síncrono. Si el email falla, se captura la excepción y se registra en el log, pero la reserva se guarda igualmente.
 """

def _notify(booking_id, notification_type, recipient):

"""Envía notificación. Intenta Celery; si no hay broker, envío directo."""

try:

from apps.notifications.tasks import send_booking_notification

try:

send_booking_notification.delay(booking_id, notification_type, recipient)

except Exception:

Celery no disponible → llamada síncrona

send_booking_notification(booking_id, notification_type, recipient)

except Exception as exc:

logger.warning('notification.failed booking=%s type=%s: %s',
 booking_id, notification_type, exc)

@receiver(post_save, sender=Booking)

def handle_booking_created_or_updated(sender, instance, created, **kwargs):

if created:

Nueva reserva → notificar al profesional

_notify(instance.id, 'booking_created', 'professional')

else:

old_status = getattr(instance, '_old_status', None)

if old_status and old_status != instance.status:

if instance.status == Booking.Status.CONFIRMED:

_notify(instance.id, 'booking_confirmed', 'client')

elif instance.status == Booking.Status.REJECTED:

_notify(instance.id, 'booking_rejected', 'client')

elif instance.status == Booking.Status.CANCELED:

```
recipient = 'professional' if instance.canceled_by == instance.client else 'client'  
_notify(instance.id, 'booking_canceled', recipient)
```

6. Implementación del Frontend

6.1 Configuración del Cliente HTTP (Axios)

El interceptor de Axios es fundamental para la experiencia de usuario: permite renovar automáticamente el token JWT sin que el usuario tenga que iniciar sesión de nuevo:

```
// src/lib/api.ts
const API_ORIGIN = process.env.NEXT_PUBLIC_API_URL || '';

export const api = axios.create({
  baseURL: API_ORIGIN,
  headers: { 'Content-Type': 'application/json' },
});

// Interceptor de PETICIÓN: añadir token JWT
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Interceptor de RESPUESTA: refresh token automático
api.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const originalRequest = error.config as any;

    // Si recibimos 401 y aún no hemos reintentado
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true; // Marcar para no entrar en bucle

      try {
        const refreshToken = localStorage.getItem('refresh_token');
        if (refreshToken) {
          // Intentar renovar con el refresh token
          const response = await axios.post(
            `${API_ORIGIN}/api/auth/token/refresh/`,
            { refresh: refreshToken }
          );

          const { access, refresh: newRefresh } = response.data;
          localStorage.setItem('access_token', access);
          // Con ROTATE_REFRESH_TOKENS=True, también se rota el refresh
          if (newRefresh) localStorage.setItem('refresh_token', newRefresh);

          // Reintentar la petición original con el nuevo token
          originalRequest.headers.Authorization = `Bearer ${access}`;
          return api(originalRequest);
        }
      } catch {
        // Refresh también falló → logout
        localStorage.removeItem('access_token');
        localStorage.removeItem('refresh_token');
        window.location.href = '/login';
      }
    }

    return Promise.reject(error);
  }
);
```

¿Por qué no usar cookies httponly?

Inicialmente se planteó usar cookies httponly para los tokens JWT (más seguro contra XSS). Sin embargo, en un monorepo Vercel con frontend + backend en el mismo dominio, hay que configurar cuidadosamente las opciones de `samesite`, `secure` y `domain`. Dado que la arquitectura ya usa `APPEND_SLASH=False` y middlewares custom para compatibilidad con Vercel, se optó por `localStorage` con las mitigaciones de seguridad habituales (CSP restrictiva, timeout corto de 15 minutos para el access token).

6.2 Wizard de Reserva en 4 Pasos

La página de reserva fue rediseñada de un formulario plano a un wizard por pasos para mejorar significativamente la experiencia de usuario. Cada paso valida y guarda el estado antes de avanzar.

```
// src/app/[locale]/booking/new/page.tsx (extracto)

// Los 4 pasos del wizard
const STEPS = ['Servicio', 'Profesional', 'Fecha y hora', 'Confirmar'];

// Estado de la selección actual
const [step, setStep] = useState(0);
const [sel, setSel] = useState({
  service: searchParams.get('service') || '', // Pre-seleccionado desde URL
  professional: searchParams.get('professional') || '',
  date: '',
  time: '', // ISO string del slot seleccionado
  notes: '',
});

// Cache para usuarios no autenticados
// Si el usuario no tiene cuenta, guardamos su selección en localStorage
// para que al registrarse continúe donde lo dejó
const goNext = () => {
  localStorage.setItem('pending_booking', JSON.stringify(sel));

  if (step === 2 && !isLoggedIn) {
    // Paso de fecha/hora requiere autenticación para confirmar
    router.push('/es/login?return=booking');
    return;
  }
  setStep(s => s + 1);
};

// El login detecta si hay reserva pendiente y redirige al formulario
// (en login/page.tsx)
useEffect(() => {
  setHasPendingBooking(!!localStorage.getItem('pending_booking'));
}, []);

// Tras hacer login exitoso:
const pendingBooking = typeof window !== 'undefined'
  ? localStorage.getItem('pending_booking')
  : null;
router.push(pendingBooking ? '/es/booking/new' : '/es/dashboard');
```

Paso 1 — Selección de servicio: Cards visuales con imagen, nombre, descripción, duración y precio. Al seleccionar, la card se resalta con borde azul y checkmark.

Paso 2 — Selección de profesional: Cards con foto del profesional, nombre, especialidad, biografía y valoración. Diseño visual que incentiva al usuario a conocer al profesional antes de reservar.

Paso 3 — Fecha y hora: Input de fecha + grid de botones de hora disponible. Los slots se cargan en tiempo real al cambiar cualquiera de las 3 variables (profesional, servicio, fecha). El slot seleccionado se resalta con animación `scale-105`.

Paso 4 — Confirmación: Resumen completo de la reserva + campo de notas opcional + botón verde "Confirmar reserva".

6.3 Pantalla de Éxito tras Reserva

```
// Pantalla de éxito - reemplaza el formulario completamente
if (success) return (
  <div className="min-h-screen bg-gradient-to-br from-green-50 to-emerald-100
    flex items-center justify-center px-4">
    <div className="max-w-md w-full text-center">
      {/* Icono de confirmación */}
      <div className="w-24 h-24 bg-gradient-to-br from-emerald-500 to-green-600
        rounded-full flex items-center justify-center mx-auto mb-6 shadow-lg">
        <CheckCircle className="w-12 h-12 text-white" />
      </div>

      <h1 className="text-3xl font-bold text-gray-900 mb-3">¡Reserva enviada!</h1>
      <p className="text-gray-600 mb-8">
        Tu solicitud ha sido enviada a <strong>{selProf?.full_name}</strong>.
        El profesional revisará tu solicitud y recibirás un email cuando sea aceptada.
      </p>

      {/* Tarjeta resumen */}
      <div className="bg-white rounded-2xl shadow-md p-6 mb-8 text-left space-y-3">
        {/* ... detalles de la reserva ... */}
      </div>

      <div className="flex gap-3 justify-center">
        <Link href="/es/bookings" className="px-6 py-3 bg-emerald-600 text-white rounded-xl">
          Ver mis reservas
        </Link>
        <Link href="/es" className="px-6 py-3 bg-white border text-gray-700 rounded-xl">
          Ir al inicio
        </Link>
      </div>
    </div>
  </div>
);
```

6.4 Gestión de Errores en Tiempo Real (Bookings)

Las acciones de confirmar/rechazar/cancelar actualizan el estado local inmediatamente, sin esperar a recargar de la API. Esto crea una experiencia más fluida:

```
// src/app/[locale]/bookings/page.tsx

// Funciones de actualización local de estado
const updateBooking = (id: number, patch: Partial<Booking>) =>
  setBookings(prev => prev.map(b => b.id === id ? { ...b, ...patch } : b));

const removeBooking = (id: number) =>
  setBookings(prev => prev.filter(b => b.id !== id));

// En BookingCard:
const doAction = async (action: () => Promise<any>, cb: (id: number) => void) => {
  setBusy(true);
  try {
    await action(); // Llamada a la API
    cb(b.id);       // Actualización del estado local
  }
}
```



```

    } catch (err: any) {
      alert(err?.response?.data?.detail || 'Error al procesar la acción.');
```

```

    } finally {
      setBusy(false);
    }
  };

// Uso:
<button onClick={() => doAction(
  () => bookingsAPI.confirm(b.id),
  (id) => updateBooking(id, { status: 'CONFIRMED' })
)}>
  Confirmar
</button>
```

6.5 Internacionalización con next-intl

```

// src/i18n/routing.ts
import { defineRouting } from 'next-intl/routing';

export const routing = defineRouting({
  locales: ['es', 'en'],
  defaultLocale: 'es',
});

// src/messages/es.json (extracto)
{
  "nav": {
    "home": "Inicio",
    "services": "Servicios",
    "professionals": "Profesionales",
    "bookings": "Reservas"
  },
  "booking": {
    "title": "Nueva Reserva",
    "step1": "Servicio",
    "step2": "Profesional",
    "step3": "Fecha y hora",
    "step4": "Confirmar",
    "success": "¡Reserva enviada!"
  }
}

// Uso en componentes:
const t = useTranslations();
<h1>{t('nav.home')}</h1>
```

7. Sistema de Seguridad

7.1 Estrategia de Autenticación JWT

Flujo completo de autenticación:

1. REGISTRO
POST /api/auth/register/
Body: { email, password1, password2, first_name, last_name, gdpr_consent: true }
Response: { access, refresh, user }
→ gdpr_consent_at se establece automáticamente
→ Se crea EmailAddress en allauth (aunque la verificación está desactivada)
2. LOGIN
POST /api/auth/login/
Body: { email, password }
Response: { access, refresh, user }
→ access: válido 15 minutos (SHORT para reducir ventana de abuso)
→ refresh: válido 7 días, se ROTA en cada uso (ROTATE_REFRESH_TOKENS=True)
→ Al rotar, el refresh anterior se añade a la blacklist (BLACKLIST_AFTER_ROTATION=True)
3. PETICIÓN AUTENTICADA
GET /api/auth/users/me/
Header: Authorization: Bearer {access_token}
→ JWTAuthentication verifica la firma y la expiración
→ Si válido: inyecta request.user
→ Si expirado: 401 Unauthorized
4. RENOVACIÓN (automática via interceptor Axios)
POST /api/token/refresh/
Body: { refresh: "{refresh_token}" }
Response: { access, refresh } ← nuevo refresh token
→ El interceptor captura el 401 y hace esta petición transparentemente
→ Si el refresh también expiró: redirect a /login
5. LOGOUT
POST /api/auth/logout/
Body: { refresh: "{refresh_token}" }
Header: Authorization: Bearer {access_token}
→ El refresh token se añade a la blacklist inmediatamente
→ El access token sigue válido 15 min (JWT es stateless por diseño)
→ Se hace flush de la sesión Django
→ Frontend borra localStorage

¿Por qué no invalidar también el access token?

El access token JWT es stateless (no se almacena en BD). Para invalidarlo habría que añadirlo también a una blacklist, lo que requiere una consulta a BD en CADA petición autenticada. Con tokens de 15 minutos, el riesgo es muy bajo y el rendimiento se mantiene.

7.2 Protección Contra Fuerza Bruta (django-axes)

```
# settings.py
AXES_ENABLED = True
AXES_FAILURE_LIMIT = 5 # 5 intentos fallidos máximo
AXES_COOLOFF_TIME = timedelta(minutes=15) # Bloqueo durante 15 minutos
AXES_RESET_ON_SUCCESS = True # El contador se resetea tras login exitoso
AXES_LOCKOUT_PARAMETERS = [ # Bloquear por combinación de usuario + IP
    ["username", "ip_address"]
]
```

```
# AUTHENTICATION_BACKENDS - axes debe ser el primero
AUTHENTICATION_BACKENDS = [
    'axes.backends.AxesStandaloneBackend',    # Verificación de bloqueo
    'django.contrib.auth.backends.ModelBackend',
    'allauth.account.auth_backends.AuthenticationBackend',
]
```

Flujo cuando se bloquea un usuario:

1. Usuario falla el login 5 veces
2. axes registra los intentos en AccessAttempt (tabla BD)
3. En el intento 6: axes devuelve respuesta de bloqueo (HTTP 429)
4. El SecureLoginView registra el bloqueo en el security.log
5. Después de 15 minutos: el bloqueo se libera automáticamente
6. Si el usuario hace login correcto antes: el contador se resetea

7.3 Content Security Policy (CSP)

La CSP es una cabecera HTTP que le dice al navegador qué orígenes son de confianza para cargar recursos. Previene ataques XSS al bloquear scripts inyectados:

```
# settings.py
CSP_DEFAULT_SRC = ('self',) # Por defecto: solo el propio dominio

# Scripts: solo cdn.jsdelivr.net (para Swagger UI) y el propio dominio
CSP_SCRIPT_SRC = ('self', 'https://cdn.jsdelivr.net', 'https://unpkg.com')

# Estilos: unsafe-inline necesario para Tailwind CSS
CSP_STYLE_SRC = ('self', 'unsafe-inline', 'https://cdn.jsdelivr.net', 'https://unpkg.com')

# Imágenes: Unsplash (imágenes de placeholder) y Cloudinary (uploads)
CSP_IMG_SRC = ('self', 'data:', 'https://images.unsplash.com',
               'https://*.vercel.app', 'https://cdn.jsdelivr.net',
               'https://res.cloudinary.com')

# Fuentes: jsdelivr para Swagger UI
CSP_FONT_SRC = ('self', 'https://cdn.jsdelivr.net', 'https://unpkg.com')

# Conexiones: Neon, Vercel y CDNs necesarios
CSP_CONNECT_SRC = ('self', 'https://*.neon.tech', 'https://*.vercel.app',
                  'https://cdn.jsdelivr.net', 'https://unpkg.com')

# Iframes: ninguno (previene clickjacking)
CSP_FRAME_ANCESTORS = ('none',)

# En producción: forzar HTTPS
CSP_UPGRADE_INSECURE_REQUESTS = not DEBUG
```

Problema real encontrado: El Swagger UI (`/api/docs/`) carga sus recursos desde `cdn.jsdelivr.net`. La CSP inicial no lo incluía, causando que la página de documentación apareciera en blanco. Se añadió `cdn.jsdelivr.net` a los orígenes permitidos para scripts, estilos, fuentes y conexiones.

7.4 Encriptación de Campos Sensibles (Fernet)

```
# Cómo funciona django-encrypted-model-fields:

# 1. Al guardar un registro:
phone_value = "+34 600 000 000"
# El campo lo transforma antes de escribir en BD:
encrypted_value = Fernet(KEY).encrypt(phone_value.encode())
# → b'gAAAAABkT...' (texto ilegible en PostgreSQL)

# 2. Al leer de la BD:
```

```

encrypted_from_db = b'gAAAAABkT...'
# El campo lo descifra automáticamente:
original_value = Fernet(KEY).decrypt(encrypted_from_db).decode()
# → "+34 600 000 000"

# Configuración de la clave:
# Si FIELD_ENCRYPTION_KEY no está configurada, se deriva del SECRET_KEY:
from cryptography.fernet import Fernet
import base64, hashlib
key = base64.urlsafe_b64encode(
    hashlib.sha256(settings.SECRET_KEY.encode()).digest()
)

```

Campos encriptados en NeoCore:

Modelo	Campo	Razón
User	phone	Dato personal sensible (GDPR)
User	bio	Información médica/personal
Booking	client_notes	Puede contener información médica
Booking	professional_notes	Información clínica confidencial
Booking	cancellation_reason	Puede revelar información sensible

8. Sistema de Emails

8.1 Problema con SMTP en Vercel

Los entornos serverless como Vercel bloquean las conexiones SMTP salientes (puertos 25, 465, 587). Esto impedía usar Gmail u otros servidores SMTP tradicionales.

Solución: Usar la **API HTTP de Resend** a través de `django-anymail`. En lugar de establecer una conexión TCP directa al servidor de correo, anymail hace una petición HTTPS a `api.resend.com`, que sí está permitida.

```
# settings.py - Configuración de email

_resend_api_key = env('RESEND_API_KEY', default='')

try:
    import anymail
    _anymail_available = True
except ImportError:
    _anymail_available = False

if _resend_api_key and _anymail_available:
    # Producción: API HTTP de Resend (funciona en Vercel)
    EMAIL_BACKEND = 'anymail.backends.resend.EmailBackend'
    ANYMAIL = {'RESEND_API_KEY': _resend_api_key}
    DEFAULT_FROM_EMAIL = env('DEFAULT_FROM_EMAIL',
                             default='NeoCore <onboarding@resend.dev>')
    INSTALLED_APPS += ['anymail']
else:
    # Desarrollo/fallback: consola (imprime en stdout)
    EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

8.2 Plantilla HTML de Recuperación de Contraseña

```
<!-- templates/account/email/password_reset_key.html -->
<!DOCTYPE html>
<html lang="es">
<head>
  <style>
    /* Estilos inline para máxima compatibilidad con clientes de email */
    .container { max-width: 600px; margin: 40px auto; }
    .header {
      background: linear-gradient(135deg, #1d4ed8 0%, #1e40af 100%);
      padding: 40px 48px; text-align: center;
    }
    .btn {
      display: inline-block;
      background: linear-gradient(135deg, #1d4ed8 0%, #1e40af 100%);
      color: #ffffff; padding: 16px 40px; border-radius: 10px;
      text-decoration: none; font-weight: 600;
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="header">
      <span class="logo-text">NeoCore</span>
    </div>
    <div class="body">
      <p class="greeting">Restablece tu contraseña</p>
      <p>Hemos recibido una solicitud para restablecer la contraseña</p>
    </div>
  </div>
</body>
</html>
```

```

de tu cuenta en <strong>NeoCore</strong>.</p>

<div class="btn-wrapper">
  <!-- {{ password_reset_url }} es el contexto de django-allauth -->
  <a href="{{ password_reset_url }}" class="btn">
    Restablecer contraseña
  </a>
</div>

<p>El enlace expirará en <strong>24 horas</strong>.</p>

<!-- Fallback por si el botón no funciona -->
<div class="fallback">
  <p>Si el botón no funciona, copia este enlace:</p>
  <a href="{{ password_reset_url }}">{{ password_reset_url }}</a>
</div>
</div>
</div>
</body>
</html>

```

8.3 URL del Reset de Contraseña apuntando al Frontend

Por defecto, allauth genera un enlace apuntando a una URL de Django (backend). Para una SPA Next.js, necesitamos que el enlace apunte al frontend:

```

# apps/users/security_views.py

def _frontend_reset_url(request, user, temp_key):
    """
    Generador de URL custom. Genera el enlace apuntando al frontend Next.js
    en lugar del backend Django.

    Resultado: https://neocoree.xyz/es/reset-password/{uid}/{token}/
    """
    from django.utils.encoding import force_bytes
    from django.utils.http import urlsafe_base64_encode
    uid = urlsafe_base64_encode(force_bytes(user.pk))
    base = getattr(settings, 'FRONTEND_URL', 'https://neocoree.xyz').rstrip('/')
    return f'{base}/es/reset-password/{uid}/{temp_key}/'

class FrontendPasswordResetSerializer(PasswordResetSerializer):
    """Usa el generador de URL custom en las opciones del formulario."""
    def get_email_options(self):
        return {
            'url_generator': _frontend_reset_url,
            'html_email_template_name': 'account/email/password_reset_key.html',
            'subject_template_name': 'account/email/password_reset_key_subject.txt',
            'email_template_name': 'account/email/password_reset_key_message.txt',
        }

```

9. Gestión de Imágenes

9.1 El Problema: Filesystem Efímero en Vercel

Las funciones serverless de Vercel tienen un filesystem efímero. El directorio `/tmp/` existe durante la vida de la instancia (minutos u horas), pero:

- Se borra cuando la función se "congela" por inactividad
- No se comparte entre instancias paralelas de la misma función
- No persiste entre despliegues

Esto significa que si se subía una imagen de perfil, podía desaparecer minutos después.

9.2 La Solución: Cloudinary

```
# settings.py

_ccloudinary_url = env('CLOUDINARY_URL', default='')

try:
    import cloudinary as _cld
    _cloudinary_pkg_ok = True
except ImportError:
    _cloudinary_pkg_ok = False

if _cloudinary_url and _cloudinary_pkg_ok:
    # Producción: almacenamiento permanente en Cloudinary CDN
    import cloudinary
    cloudinary.config(url=_cloudinary_url, secure=True)
    DEFAULT_FILE_STORAGE = 'cloudinary_storage.storage.MediaCloudinaryStorage'
    CLOUDINARY_STORAGE = {'CLOUDINARY_URL': _cloudinary_url}
    MEDIA_URL = '/media/'
    MEDIA_ROOT = BASE_DIR / 'media'
    INSTALLED_APPS += ['cloudinary_storage', 'cloudinary']
else:
    # Fallback: filesystem local (efímero en Vercel, persistente en Docker)
    MEDIA_URL = '/media/'
    MEDIA_ROOT = BASE_DIR / 'media' if not _is_vercel else Path('/tmp/media')
    MEDIA_ROOT.mkdir(parents=True, exist_ok=True)
```

Cómo funciona con Cloudinary:

1. El frontend envía la imagen con `multipart/form-data` al backend
2. Django recibe el archivo con el `ImageField`
3. `django-cloudinary-storage` intercepta el guardado y sube a Cloudinary
4. La URL devuelta es una URL de Cloudinary CDN:
`https://res.cloudinary.com/cloud_name/image/upload/...`
5. Esta URL es permanente y se sirve desde la CDN global de Cloudinary

Sin Cloudinary (fallback):

El rewrite de Vercel `/media/*` → Django permite que las imágenes se sirvan desde `/tmp/media/` mientras la instancia esté activa. Es válido para demos y desarrollo.

9.3 Validación de Imágenes en el Serializer

```

# apps/users/serializers.py
ALLOWED_IMAGE_TYPES = ['image/jpeg', 'image/png', 'image/webp']

class UserUpdateSerializer(serializers.ModelSerializer):

    def validate_profile_image(self, value):
        """
        Valida el tamaño y tipo MIME de la imagen de perfil.
        Usa python-magic si está disponible; fallback por extensión.
        """
        if not value:
            return value

        # Límite de tamaño: 2 MB
        if value.size > 2 * 1024 * 1024:
            raise serializers.ValidationError(
                'La imagen no puede superar 2 MB.'
            )

        # Validar tipo MIME
        try:
            import magic
            mime_type = magic.from_buffer(value.read(1024), mime=True)
            value.seek(0)
            if mime_type not in ALLOWED_IMAGE_TYPES:
                raise serializers.ValidationError(
                    f'Tipo de imagen no permitido: {mime_type}. '
                    f'Usa JPEG, PNG o WebP.'
                )
        except ImportError:
            # python-magic no disponible en Vercel (libmagic no compilada)
            # Fallback: verificar por extensión del archivo
            ext = value.name.split('.')[-1].lower()
            if ext not in ['jpg', 'jpeg', 'png', 'webp']:
                raise serializers.ValidationError(
                    'Usa archivos JPEG, PNG o WebP.'
                )

        return value

```


10. API REST — Documentación Completa

10.1 Principios de Diseño de la API

La API sigue los principios REST: - **Recursos como substantivos** en las URLs (`/api/bookings/`, `/api/services/`) - **Verbos HTTP** para acciones (GET, POST, PATCH, DELETE) - **Códigos de estado semánticos** (200, 201, 400, 401, 403, 404, 429) - **JSON** como formato de intercambio - **Paginación** en listados (20 elementos/página) - **Filtrado y ordenación** mediante query params - **Versiónado** en la URL base (`/api/`)

10.2 Endpoints de Autenticación

POST `/api/auth/register/`

```
POST /api/auth/register/
Content-Type: application/json

{
  "email": "cliente@ejemplo.com",
  "password1": "MiContraseña123!",
  "password2": "MiContraseña123!",
  "first_name": "Juan",
  "last_name": "García",
  "phone": "+34 600 000 000", // Opcional
  "gdpr_consent": true      // Obligatorio
}
```

Respuesta 201 Created:

```
{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "email": "cliente@ejemplo.com",
    "first_name": "Juan",
    "last_name": "García",
    "full_name": "Juan García",
    "role": "CLIENT",
    "phone": "+34600000000",
    "specialty": "",
    "bio": "",
    "profile_image": null,
    "gdpr_consent": true,
    "gdpr_consent_at": "2026-05-06T10:30:00+02:00",
    "is_active": true,
    "created_at": "2026-05-06T10:30:00+02:00"
  }
}
```

Validaciones: - Email único (caso insensible) - Contraseña mínimo 8 caracteres, no demasiado común, no solo numérica - `first_name` y `last_name` solo letras (incluye acentos, ñ, guión) - `gdpr_consent` debe ser `true` - Rate limit: 10 registros/hora por IP

POST `/api/auth/login/`

```
POST /api/auth/login/
Content-Type: application/json

{
  "email": "cliente@ejemplo.com",
  "password": "MiContraseña123!"
}
```

Rate limit: 5 intentos/hora por IP. Tras 5 fallos: bloqueo de 15 minutos (django-axes).

10.3 Endpoints de Reservas

POST /api/bookings/ — Crear reserva

```
POST /api/bookings/
Authorization: Bearer {access_token}
Content-Type: application/json

{
  "service": 1,
  "professional": 2,
  "start_datetime": "2026-06-15T10:00:00Z",
  "end_datetime": "2026-06-15T11:00:00Z",
  "client_notes": "Primera visita, tengo dolor lumbar desde hace 2 semanas."
}
```

Validaciones del serializer:

```
def validate(self, attrs):
    """
    Validaciones de negocio para crear una reserva.
    """
    start = attrs['start_datetime']
    end = attrs['end_datetime']

    # 1. La hora de fin debe ser posterior a la de inicio
    if end <= start:
        raise ValidationError('La hora de fin debe ser posterior a la de inicio.')

    # 2. No se pueden hacer reservas en el pasado
    if start < timezone.now():
        raise ValidationError('No se pueden crear reservas en el pasado.')

    # 3. El slot debe estar dentro de la disponibilidad del profesional
    professional = attrs['professional']
    day_of_week = start.weekday()
    start_time = start.time()
    end_time = end.time()

    rule = AvailabilityRule.objects.filter(
        professional=professional,
        day_of_week=day_of_week,
        start_time__lte=start_time,
        end_time__gte=end_time,
        is_active=True,
    ).first()

    if not rule:
        raise ValidationError("Slot not within professional's availability")

    # 4. No puede solaparse con otra reserva
    overlapping = Booking.objects.filter(
        professional=professional,
        status__in=[Booking.Status.PENDING, Booking.Status.CONFIRMED],
        start_datetime__lt=end,
        end_datetime__gt=start,
```

```

).exclude(id=self.instance.id if self.instance else None)

if overlapping.exists():
    raise ValidationError('Slot overlaps with existing booking')

# 5. No puede ser durante una ausencia
time_off = TimeOff.objects.filter(
    professional=professional,
    start_date__lte=start.date(),
    end_date__gte=start.date(),
).first()

if time_off:
    raise ValidationError('Professional is on time off during this period')

return attrs

```

GET /api/availability/slots/get_slots/ — Slots disponibles

GET /api/availability/slots/get_slots/?professional_id=2&service_duration=60&start_date=2026-06-10&end_date=2026-06-15

Respuesta:

```

{
  "slots": [
    {
      "start_datetime": "2026-06-10T09:00:00+02:00",
      "end_datetime": "2026-06-10T10:00:00+02:00",
      "is_available": true
    },
    {
      "start_datetime": "2026-06-10T10:00:00+02:00",
      "end_datetime": "2026-06-10T11:00:00+02:00",
      "is_available": true
    }
    // ... más slots
  ]
}

```

10.4 Documentación Swagger/OpenAPI

La API está completamente documentada con **drf-spectacular**, accesible en </api/docs/>. Configuración:

```

SPECTACULAR_SETTINGS = {
    'TITLE': 'NeoCore API',
    'VERSION': '1.0.0',
    'SECURITY': [{ 'jwtAuth': [] }],
    'SECURITY_DEFINITIONS': {
        'jwtAuth': {
            'type': 'http',
            'scheme': 'bearer',
            'bearerFormat': 'JWT',
        },
    },
    'SWAGGER_UI_SETTINGS': {
        'persistAuthorization': True,      # Token persiste al recargar
        'displayRequestDuration': True,   # Muestra tiempo de respuesta
        'tryItOutEnabled': True,          # Prueba activada por defecto
    },
}

```

Para autenticar en Swagger UI: 1. Hacer POST /api/auth/login/ → copiar el valor de `access` 2. Pulsar el botón **Authorize** (candado, arriba a la derecha) 3. Pegar el token 4. Todos los endpoints ahora se

ejecutarán autenticados

11. Despliegue en Producción

11.1 Configuraciones Específicas de Vercel

Django detecta si está en Vercel mediante la variable de entorno `VERCEL=1` y ajusta su comportamiento:

```
# settings.py
_is_vercel = bool(env('VERCEL', default=''))

if _is_vercel:
    # El filesystem es read-only excepto /tmp
    MEDIA_ROOT = Path('/tmp/media')

    # WhiteNoise en modo finder (sin collectstatic previo)
    WHITENOISE_USE_FINDERS = True
    STATICFILES_STORAGE = 'whitenoise.storage.CompressedStaticFilesStorage'

    # Vercel maneja HTTPS en el edge, Django no hace redirect
    SECURE_SSL_REDIRECT = False

    # Solo JSON (sin browsable API que viola CSP)
    REST_FRAMEWORK = {
        'DEFAULT_RENDERER_CLASSES': ['rest_framework.renderers.JSONRenderer'],
        # ...
    }
```

11.2 Pipeline de CI/CD

El despliegue es automático: cada `git push origin main` dispara un despliegue en Vercel.

1. Developer hace push a GitHub
 - └─ `git push origin main`
2. GitHub notifica a Vercel (webhook)
3. Vercel inicia el build:
 - └─ `npm install (root + frontend)`
 - └─ `cd frontend && npm run build (Next.js)`
 - └─ Python: `pip install -r requirements.txt`
4. Vercel despliega:
 - └─ Frontend: páginas estáticas en CDN
 - └─ Frontend: rutas dinámicas como Edge Functions
 - └─ Backend: `api/index.py` como Función Python serverless
5. Vercel actualiza el dominio `neocoree.xyz`
 - └─ Sin downtime (atomic deployments)

11.3 Seed de Datos en Producción

Para cargar datos de prueba realistas en producción:

```
# Llama al endpoint de seed (protegido por SECRET_KEY)
curl -X POST https://neocoree.xyz/api/seed \
  -H "X-Seed-Secret: TU_SECRET_KEY_DE_VERCEL"

# El comando seed_data crea:
# - 1 admin: admin@neocore.com / NeoCore2026!
# - 12 profesionales: Prof2026! (Fisio, Nutrición, Psicología, Cardiología...)
```

```
# - 220+ clientes: Client2026!  
# - 10 servicios (con distintas duraciones y precios)  
# - Reglas de disponibilidad para cada profesional  
# - 80 reservas en distintos estados
```

11.4 Variables de Entorno en Producción

Variable	Descripción	Dónde obtenerla
SECRET_KEY	Clave secreta Django	Generar con <code>python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"</code>
DATABASE_URL	URL Neon PostgreSQL	Dashboard de Neon.tech
DEBUG	Siempre <code>False</code>	—
VERCEL	Siempre <code>1</code>	—
RESEND_API_KEY	API key de email	Dashboard de Resend.com
CLOUDINARY_URL	URL de Cloudinary	Dashboard de Cloudinary
FRONTEND_URL	URL del frontend	<code>https://neocoree.xyz</code>
ADMIN_PATH	Ruta del admin real	Ej: <code>mi-admin-secreto/</code>

12. Pruebas y Validación

12.1 Tests de la API en Producción

Durante el desarrollo se realizaron tests exhaustivos de todos los endpoints usando `curl`. A continuación se muestran los resultados del test de aceptación final:

```
# Test 1: Registro de usuario
POST /api/auth/register/ → 201 Created ✓
Validación email duplicado → 400 {"email": ["Ya existe..."]} ✓
Validación GDPR → 400 {"gdpr_consent": ["Debes aceptar..."]} ✓
Validación nombre con números → 400 ✓

# Test 2: Autenticación
POST /api/auth/login/ → 200 + access + refresh ✓
Login con contraseña incorrecta → 400 ✓
Refresh de token → 200 + nuevo access ✓
Logout (blacklist) → 200 ✓

# Test 3: Perfil
GET /api/auth/users/me/ → 200 + datos del usuario ✓
GET /api/auth/users/me/ sin token → 401 ✓
PATCH /api/auth/users/update_me/ → 200 + datos actualizados ✓

# Test 4: Servicios (público)
GET /api/services/ → 200 + lista de servicios ✓
GET /api/services/?search=fisio → 200 + filtrado ✓

# Test 5: Disponibilidad
GET /api/availability/slots/get_slots/?professional_id=26&service_duration=60 → 200 + slots ✓
Sin professional_id → 400 ✓

# Test 6: Reservas
POST /api/bookings/ (cliente) → 201 + booking PENDING ✓
POST /api/bookings/ (admin) → 403 "Only clients can create" ✓
POST /api/bookings/{id}/confirm/ (profesional) → 200 + CONFIRMED ✓
POST /api/bookings/{id}/reject/ (profesional) → 200 + REJECTED ✓

# Test 7: Estadísticas (admin)
GET /api/bookings/stats/?days=30 → 200 + stats ✓
GET /api/bookings/stats/ (sin auth) → 401 ✓
GET /api/bookings/stats/ (con token cliente) → 403 ✓

# Test 8: Páginas frontend
GET /es/ → 200 ✓
GET /es/services → 200 ✓
GET /es/login → 200 ✓
GET /es/backoffice/api → 200 ✓
GET /es/privacy → 200 ✓
GET /es/pagina-inexistente → 404 ✓
```

12.2 Verificación de Seguridad

```
# Test SQL Injection
POST /api/auth/login/ -d '{"email": "x\' OR 1=1 --", "password": "x"}'
→ 400 {"email": "Invalid"} ✓ (bloqueado por serializer)

curl "https://neocoree.xyz/api/bookings/?search=1 UNION SELECT password FROM users"
→ 400 (bloqueado por SQLInjectionProtectionMiddleware) ✓

# Test XSS
PATCH /api/auth/users/update_me/ -d '{"first_name": "<script>alert(1)</script>"}'
```

```

→ 400 {"first_name": "El nombre contiene caracteres no permitidos."} ✓

# Test Rate Limiting (login)
for i in 1..6; do POST /api/auth/login/ con contraseña incorrecta; done
→ Intentos 1-5: 400
→ Intento 6: 429 (bloqueado por django-axes) ✓

# Test CSP headers
curl -I https://neocoree.xyz/api/health/
→ content-security-policy: frame-ancestors 'none'; script-src 'self'
https://cdn.jsdelivr.net ... ✓

# Test HTTPS / HSTS
curl -I https://neocoree.xyz
→ strict-transport-security: max-age=31536000; includeSubDomains; preload ✓

```

12.3 Tests Unitarios del Backend

```

# apps/bookings/tests/test_availability.py
import pytest
from apps.availability.services import AvailabilityService

@pytest.mark.django_db
class TestAvailabilityService:

    def test_no_slots_without_rules(self, professional):
        """Sin reglas de disponibilidad, no hay slots."""
        slots = AvailabilityService.get_available_slots(
            professional_id=professional.id,
            service_duration=60,
            start_date=date.today(),
            end_date=date.today() + timedelta(days=7),
        )
        assert len(slots) == 0

    def test_slots_generated_correctly(self, professional, availability_rule):
        """Con regla L-V 9-18, debe generar 9 slots de 1h por día."""
        monday = get_next_monday()
        slots = AvailabilityService.get_available_slots(
            professional_id=professional.id,
            service_duration=60,
            start_date=monday,
            end_date=monday, # Solo lunes
        )
        assert len(slots) == 9 # 9:00, 10:00, ..., 17:00

    def test_booked_slot_excluded(self, professional, availability_rule, booking):
        """Un slot ya reservado (CONFIRMED) no debe aparecer."""
        slots = AvailabilityService.get_available_slots(
            professional_id=professional.id,
            service_duration=60,
            start_date=booking.start_datetime.date(),
            end_date=booking.start_datetime.date(),
        )
        booked_times = [s['start_datetime'] for s in slots]
        assert booking.start_datetime.isoformat() not in booked_times

    def test_time_off_excludes_day(self, professional, availability_rule, time_off):
        """Durante un período de ausencia, no hay slots."""
        slots = AvailabilityService.get_available_slots(
            professional_id=professional.id,
            service_duration=60,
            start_date=time_off.start_date,
            end_date=time_off.end_date,
        )
        assert len(slots) == 0

```


13. Conclusiones y Trabajo Futuro

13.1 Conclusiones

NeoCore ha sido desarrollado como un sistema web completo y funcional que aborda un problema real del sector salud y bienestar. A lo largo del proyecto se han aplicado y profundizado los conocimientos adquiridos durante el ciclo formativo DAW, extendidos con tecnologías modernas de la industria.

Logros técnicos destacados:

1. **Arquitectura serverless en producción:** La adaptación de Django a un entorno serverless (Vercel) fue uno de los mayores retos. Problemas como la gestión de migraciones en cold start, el filesystem efímero, los rewrites de Vercel y la compatibilidad del middleware con el pipeline de DRF requirieron soluciones creativas y profundo conocimiento del stack.
2. **Seguridad multicapa:** Se implementaron más de 10 capas de seguridad independientes (JWT, Argon2, Axes, CSP, CORS, middleware SQL injection, encriptación de campos, honeypot). Cada una cubre un vector de ataque diferente, siguiendo el principio de "defensa en profundidad".
3. **Motor de disponibilidad:** El algoritmo de cálculo de slots disponibles gestiona correctamente disponibilidad semanal, reservas existentes, ausencias y zonas horarias, generando respuestas en milisegundos.
4. **Experiencia de usuario:** El wizard de reserva de 4 pasos, el cache para usuarios no autenticados, las actualizaciones de estado en tiempo real y las pantallas de error personalizadas crean una experiencia fluida y profesional.
5. **Calidad del código:** La arquitectura modular de Django con ViewSets, serializers separados por rol, permissions reutilizables y signals para desacoplar la lógica de negocio facilita el mantenimiento y la escalabilidad.

13.2 Dificultades Encontradas

1. **Vercel + Django + trailing slashes:** El CDN de Vercel elimina los trailing slashes antes de llegar a Django, causando 404. Solución: `TrailingSlashMiddleware` con reescritura interna sin HTTP redirect.
2. **JWT_AUTH_HTTPONLY en dj-rest-auth 5.x:** Esta biblioteca cambia el comportamiento por defecto en la versión 5: el refresh token no se incluye en la respuesta JSON si `JWT_AUTH_HTTPONLY=True` (por defecto). La solución fue configurar `JWT_AUTH_HTTPONLY=False` explícitamente.
3. **django-ratelimit vs DRF pipeline:** El decorador `@method_decorator(ratelimit(...), name='dispatch')` corrompe el pipeline de renderizado de DRF. Solución: usar el throttling nativo de DRF con `throttle_classes`.
4. **SMTP bloqueado en Vercel:** Las funciones serverless de Vercel no pueden establecer conexiones TCP a servidores SMTP. Solución: usar la API HTTP de Resend mediante `django-anymail`.
5. **SSR + localStorage:** Acceder a `localStorage` directamente en el render de un componente Next.js causa errores en SSR (el servidor no tiene `window`). Solución: mover todas las lecturas de

`localStorage` a efectos `useEffect` o eventos de usuario.

13.3 Trabajo Futuro

Funcionalidades pendientes:

1. **Pagos online:** Integración con Stripe para cobrar las citas en el momento de la reserva, con soporte para reembolsos automáticos en caso de cancelación.
 2. **App móvil:** Desarrollar una app nativa (React Native) para iOS y Android que consuma la misma API REST, con notificaciones push.
 3. **Videoconsulta:** Integración con Daily.co o Jitsi para consultas telemáticas, con sala virtual generada automáticamente al confirmar una cita.
 4. **Recordatorios automáticos:** El sistema de Celery Beat está preparado; solo falta configurar el broker Redis en producción para activar los recordatorios de 24h antes de la cita.
 5. **Panel de analytics avanzado:** Gráficas de evolución temporal, tasa de conversión, ingresos por servicio y profesional, predicción de demanda.
 6. **Sistema de reseñas público:** Las reseñas ya existen en el modelo y backend, pero falta la interfaz pública donde los clientes puedan dejar valoraciones visibles en el perfil del profesional.
 7. **Soporte multiempresa:** Permitir que múltiples centros de salud compartan la plataforma con tenants separados.
-

14. Glosario

Término	Definición
API REST	Arquitectura de servicios web que usa HTTP y URLs para comunicar sistemas
JWT	JSON Web Token: token compacto y firmado para autenticación stateless
CSRF	Cross-Site Request Forgery: ataque que fuerza acciones no deseadas al usuario
CSP	Content Security Policy: cabecera HTTP que restringe los orígenes de recursos
XSS	Cross-Site Scripting: inyección de scripts maliciosos en páginas web
SQL Injection	Ataque que inyecta código SQL malicioso en consultas de base de datos
Argon2	Algoritmo de hash de contraseñas ganador del Password Hashing Competition 2015
Fernet	Esquema de cifrado simétrico autenticado basado en AES-128-CBC
Serverless	Arquitectura donde el proveedor gestiona la infraestructura; escala automáticamente
Cold Start	Primera invocación de una función serverless que requiere inicialización
GDPR	General Data Protection Regulation: normativa europea de protección de datos
SSR	Server-Side Rendering: renderizado de páginas en el servidor (Next.js)
CDN	Content Delivery Network: red global de servidores que acelera la entrega de contenido
ORM	Object-Relational Mapping: abstracción que permite consultar la BD con objetos Python
ViewSet	Clase de DRF que agrupa las operaciones CRUD de un recurso
Serializer	Clase que convierte objetos Python a/desde JSON, con validaciones
Middleware	Componente que procesa todas las peticiones HTTP antes de llegar a las vistas
Celery	Sistema de cola de tareas asíncronas para Python
Throttling	Limitación de la tasa de peticiones para prevenir abuso
Honeypot	Trampa para atacantes: URL falsa que registra accesos no autorizados

15. Bibliografía y Referencias

Documentación oficial

1. **Django Documentation** — <https://docs.djangoproject.com/en/4.2/>
2. **Django REST Framework** — <https://www.django-rest-framework.org/>
3. **Next.js Documentation** — <https://nextjs.org/docs>
4. **dj-rest-auth** — <https://dj-rest-auth.readthedocs.io/>
5. **drf-spectacular** — <https://drf-spectacular.readthedocs.io/>
6. **Vercel Documentation** — <https://vercel.com/docs>
7. **Neon Documentation** — <https://neon.tech/docs>
8. **Resend Documentation** — <https://resend.com/docs>
9. **Cloudinary Documentation** — <https://cloudinary.com/documentation>

Seguridad y buenas prácticas

10. **OWASP Top 10** — <https://owasp.org/www-project-top-ten/>
11. **OWASP REST Security Cheat Sheet** — https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html
12. **GDPR — Reglamento General de Protección de Datos** — <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX%3A32016R0679>
13. **Django Security** — <https://docs.djangoproject.com/en/4.2/topics/security/>
14. **JWT Best Practices** — <https://auth0.com/blog/a-look-at-the-latest-draft-for-jwt-bcp/>
15. **Argon2 RFC 9106** — <https://www.rfc-editor.org/rfc/rfc9106>

Herramientas y librerías

16. **Tailwind CSS** — <https://tailwindcss.com/docs>
17. **Radix UI** — <https://www.radix-ui.com/>
18. **next-intl** — <https://next-intl-docs.vercel.app/>
19. **django-axes** — <https://django-axes.readthedocs.io/>
20. **django-encrypted-model-fields** — <https://github.com/foundertherapy/django-encrypted-model-fields>
21. **django-anymail** — <https://anymail.dev/>
22. **django-cloudinary-storage** — <https://github.com/klis87/django-cloudinary-storage>

Documento generado como parte del Trabajo de Fin de Grado — Técnico Superior en Desarrollo de Aplicaciones Web — Ciclo 2025-2026

Autor: Raúl López | raullopez20r@gmail.com | <https://neocoree.xyz>